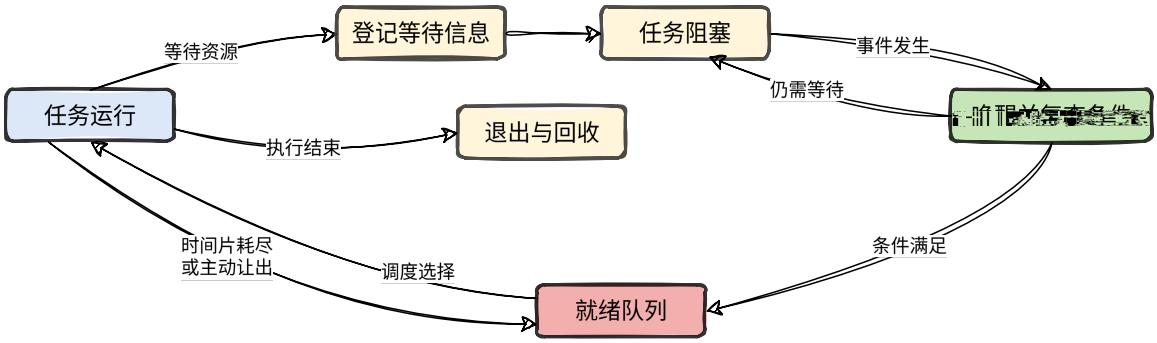
3

CPU moss

StarryOS



3-1

StarryOS moss

3.1

3.1.1 StarryOS

StarryOS `axtask`

`axsched` RR CPU

CPU

StarryOS Future Waker I/O

Futex WAIT

WAKE robust list

moss

3.1.2 StarryOS

“ Waker ” Futex

Waker

`FUTEX_WAIT_BITSET`

Waker

3.1.3 moss

moss StarryOS RR

	StarryOS	moss
	RR CPU	
	Future Waker	PollSet
		TimerKey
Futex	WAIT WAKE	
		Waker

3-1

CPU CPU

Waker

Futex

3.2

3.2.1

- `Running` CPU
- `Ready`
- `Blocked`
- `Exited`

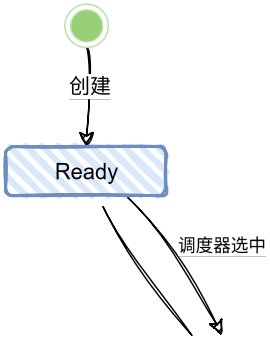
```rust
pub enum TaskState {
    Running = 1, // 正在 CPU 上  行
    Ready   = 2, // 可以  行，正在  列中等待
    Blocked = 3, // 正在等待事件，    不能  行
    Exited  = 4, // 已    束，等待  源回收
}

pub struct TaskInner {
    //
    state: AtomicU8,
    // ...
}
```

3-1

`state` Futex

CPU `Blocked` `Ready`



3.2.2

RR CPU CPU

Ready 5

CPU

scheduler_timer_tick



CPU

CPU idle

3.2.3

resched CPU

idle CPU

```
fn resched(&mut self) {
    // 从就    列    任  ;  列  空    行 idle 任
    let next = self.scheduler.lock().pick_next_task()
        .unwrap_or_else(|| unsafe {
            IDLE_TASK.current_ref_raw().get_unchecked().clone()
        });

    self.switch_to(crate::current(), next);
}

fn switch_to(&mut self, prev: CurrentTask, next: AxTaskRef) {
    // 将  中的任      正在  行
    next.set_state(TaskState::Running);

    // 保存当前任      , 恢复下一个任
    let prev_ctx = prev.ctx_mut_ptr();
    let next_ctx = next.ctx_mut_ptr();
    unsafe { (*prev_ctx).switch_to(&*next_ctx) };
}
```

3-2

switch_to Running “ ”

TaskContext

CPU TaskContext

moss TaskContext

UserContext

3.2.2

→ → → →

3.2.4 CPU

CPU cpumask CPU

CPU 0 CPU 2 CPU

CPU cpumask

CPU CPU

```
fn select_run_queue_index(cpumask: AxCpuMask) -> usize {
    loop {
        // 在所有 CPU
        let index = RUN_QUEUE_INDEX.fetch_add(1, Ordering::SeqCst)
            % axconfig::plat::CPU_NUM;
        // 只      和性掩  允  的 CPU
        if cpumask.get(index) {
            return index;
        }
    }
}
```

3-3 CPU

CPU CPU

moss

CPU moss

CPU 1

Linux sched_getaffinity sched_setaffinity

```
// 用  位  中的每一位    一个 CPU
for cpu_id in 0..axconfig::plat::CPU_NUM {
    if user_mask[cpu_id / 8] & (1 << (cpu_id % 8)) != 0 {
        cpu_mask.set(cpu_id, true);
    }
}

// 保存新的  和性范
let curr = current().clone();
curr.set_cpumask(cpu_mask);

// 当前 CPU 已不在范  内  ，将任    移到允  的 CPU
if !cpu_mask.get(axhal::percpu::this_cpu_id()) {
    let migration_task = TaskInner::new(
        move || migrate_entry(curr),
        "migration-task".into(),
        4096,
    ).into_arc();
    current_run_queue().migrate_current(migration_task);
}
```

3-4 CPU

CPU CPU

moss

`sched_yield` CPU moss

3.3

3.3.1

`Blocked`

moss Future “ ” Waker “

”

Waker

Future “ ” Waker

→ Waker → →

→ Waker → Future →

3-3

Waker

Waker

Waker “ ”

3.3.2 Waker PollSet

Waker StarryOS

Waker `wake()` moss

timerfd pidfd Futex

`PollSet` Waker eventfd timerfd pidfd

moss `has_waiters` Waker

`wake()`

```rust
pub struct PollSet {
    inner: Lazy<Mutex<Inner>>,
    // 没有等待者 ，wake 可以直接返回
    has_waiters: AtomicBool,
}

pub fn wake(&self) -> usize {
    if !self.has_waiters.load(Ordering::Acquire) {
        return 0;
    }
    // 取出并清空本 登 的 Waker
    let (wakers, len) = self.inner.lock().take_wakers();
    self.has_waiters.store(false, Ordering::Release);
    // 真正 醒 已 不再持有内部
    for waker in wakers.iter().take(len) {
        unsafe { waker.assume_init_read() }.wake();
    }
    len
}
```

3-5 PollSet

`take_wakers` Waker

PollSet “ ”

Futex

3.3.3 block_on

Future

`block_on` Future `Ready` `Pending`

CPU 3.2.3

```rust
fn wake_by_ref(self: &Arc<Self>) {
    if let Some(task) = self.task.upgrade() {
        // 先    通知已  到
        *self.woke.lock() = true;
        // 再把阻塞任  放回合适的  行  列
        select_run_queue(&task).unblock_task(task, false);
    }
}

match future.poll(&mut context) {
    // 没收到通知，当前任  可以安全  入 Blocked
    Poll::Pending if !*woke => run_queue.blocked_resched(woke),
    // 通知先到，不再阻塞，只  其他任  先  行一次
    Poll::Pending => yield_now(),
    Poll::Ready(output) => return output,
}
```

3-6 Waker block_on

Future `Pending` Waker `woke` `Blocked` CPU `unblock_task` `woke` “ ” Future `interruptible` Future `EINTR` Waker PollSet Waker Futex 3.5

3.4

3.3 “ Waker ”

3.4.1

“ ” “ ” `FUTEX_WAIT_BITSET`

nanosleep clock_nanosleep

TIMER_ABSTIME moss

EINTR

3.4.2

StarryOS

moss BTreeMap

deadline seq TimerKey

```
struct TimerKey {
    // 定  事件  当触  的  刻
    deadline: Duration,
    // 区分截止    相同的多个事件
    seq: u64,
}

struct AlarmRuntime {
    //
    next_seq: u64,
    // 按到期  序保存全部定  事件
    timers: BTreeMap<TimerKey, Entry>,
}
```

3-7

seq

TimeManager

ITIMER_REAL SIGALRM ITIMER_VIRTUAL

ITIMER_PROF

3.4.3

wait

Waker child_exit_event

```
// 先    是否已有子  程状  可以  告
match check_children().transpose() {
    Some(result) => Poll::Ready(result),
    None => {
        //                      Waker
        child_exit_event.register(cx.waker());
        // 登  后复  ，覆盖同    生的状    化
        match check_children().transpose() {
            Some(result) => Poll::Ready(result),
            None => Poll::Pending,
        }
    }
}
```

3-8

Waker

`check_children`

3.3.1 “ Waker ”

3.5 Futex

3.5.1

Futex

moss `FutexKey` Futex

Futex

Futex `owner_dead`

moss

```
pub struct WaitQueue {
    // 每个等待  依次保存：
    // 通知状  、  醒位掩  和任    醒入口
    queue: SpinNoIrq<
        VecDeque<(Arc<AtomicBool>, u32, Arc<PollSet>)>
    >,
}
```

3-9 Futex

`AtomicBool` `notified` `u32` bitset

`PollSet` `notified`

PollSet

3.5.2 WAIT WAKE

EAGAIN

Future FUTEX_WAKE

notified PollSet

wake

```
// 第一次 ：通知可能已 先于登 到
if notified.load(Ordering::Acquire) {
    return Poll::Ready(());
}

// 条件未 足，登 当前任 的 Waker
poll.register(cx.waker());

// 第二次 ：覆盖“ ”的 争窗口
if notified.load(Ordering::Acquire) {
    Poll::Ready(())
} else {
    Poll::Pending
}
```

3-10 Futex Waker

3.3.1 Waker

notified notified PollSet Waker

3.5.3

notified FUTEX_WAIT_BITSET

FUTEX_WAKE_BITSET bitset mask

FUTEX_REQUEUE Futex

FUTEX_CMP_REQUEUE EAGAIN

notified bitset PollSet Waker

pthread

3.5.4

Futex moss

interruptible(timeout(...)) ETIMEDOUT EINTR

notified